第7章

取消与关闭

任务和线程的启动很容易。在大多数时候，我们都会让它们运行直到结束，或者让它们自行停止。然而，有时候我们希望提前结束任务或线程，或许是因为用户取消了操作，或者应用程序需要被快速关闭。

要使任务和线程能安全、快速、可靠地停止下来，并不是一件容易的事。Java 没有提供任何机制来安全地终止线程[⊖]。但它提供了中断（Interruption），这是一种协作机制，能够使一个线程终止另一个线程的当前工作。

这种协作式的方法是必要的，我们很少希望某个任务、线程或服务立即停止，因为这种立即停止会使共享的数据结构处于不一致的状态。相反，在编写任务和服务时可以使用一种协作的方式：当需要停止时，它们首先会清除当前正在执行的工作，然后再结束。这提供了更好的灵活性，因为任务本身的代码比发出取消请求的代码更清楚如何执行清除工作。

生命周期结束（End-of-Lifecycle）的问题会使任务、服务以及程序的设计和实现等过程变得复杂，而这个在程序设计中非常重要的要素却经常被忽略。一个在行为良好的软件与勉强运行的软件之间的最主要区别就是，行为良好的软件能很完善地处理失败、关闭和取消等过程。本章将给出各种实现取消和中断的机制，以及如何编写任务和服务，使它们能对取消请求做出响应。

7.1 任务取消

如果外部代码能在某个操作正常完成之前将其置入“完成”状态，那么这个操作就可以称为可取消的（Cancellable）。取消某个操作的原因很多：

用户请求取消。用户点击图形界面程序中的“取消”按钮，或者通过管理接口来发出取消请求，例如 JMX（Java Management Extensions）。

有时间限制的操作。例如，某个应用程序需要在有限时间内搜索问题空间，并在这个时间内选择最佳的解决方案。当计时器超时时，需要取消所有正在搜索的任务。

应用程序事件。例如，应用程序对某个问题空间进行分解并搜索，从而使不同的任务可以搜索问题空间中的不同区域。当其中一个任务找到了解决方案时，所有其他仍在搜索的任务都

⊖ 虽然 Thread.stop 和 suspend 等方法提供了这样的机制，但由于存在着一些严重的缺陷，因此应该避免使用。请参见 http://java.sun.com/j2se/1.5.0/docs/guide/misc/threadPrimitiveDeprecation.html 了解对这些问题的详细说明。

将被取消。

错误。网页爬虫程序搜索相关的页面，并将页面或摘要数据保存到硬盘。当一个爬虫任务发生错误时（例如，磁盘空间已满），那么所有搜索任务都会取消，此时可能会记录它们的当前状态，以便稍后重新启动。

关闭。当一个程序或服务关闭时，必须对正在处理和等待处理的工作执行某种操作。在平缓的关闭过程中，当前正在执行的任务将继续执行直到完成，而在立即关闭过程中，当前的任务则可能取消。

在 Java 中没有一种安全的抢占式方法来停止线程，因此也就没有安全的抢占式方法来停止任务。只有一些协作式的机制，使请求取消的任务和代码都遵循一种协商好的协议。

其中一种协作机制能设置某个“已请求取消（Cancellation Requested）”标志，而任务将定期地查看该标志。如果设置了这个标志，那么任务将提前结束。程序清单 7-1 中就使用了这项技术，其中的 PrimeGenerator 持续地枚举素数，直到它被取消。cancel 方法将设置 cancelled 标志，并且主循环在搜索下一个素数之前会首先检查这个标志。（为了使这个过程能可靠地工作，标志 cancelled 必须为 volatile 类型。）

程序清单 7-1　使用 volatile 类型的域来保存取消状态

```
@ThreadSafe
public class PrimeGenerator implements Runnable {
    @GuardedBy("this")
    private final List<BigInteger> primes
            = new ArrayList<BigInteger>();
    private  volatile boolean cancelled;

    public void run() {
        BigInteger p = BigInteger.ONE;
        while (!cancelled ) {
            p = p.nextProbablePrime();
            synchronized (this) {
                primes.add(p);
            }
        }
    }

    public void cancel() { cancelled = true;  }

    public synchronized List<BigInteger> get() {
        return new ArrayList<BigInteger>(primes);
    }
}
```

程序清单 7-2 给出了这个类的使用示例，即让素数生成器运行 1 秒钟后取消。素数生成器通常并不会刚好在运行 1 秒钟后停止，因为在请求取消的时刻和 run 方法中循环执行下一次检查之间可能存在延迟。cancel 方法由 finally 块调用，从而确保即使在调用 sleep 时被中断也能取消素数生成器的执行。如果 cancel 没有被调用，那么搜索素数的线程将永远运行下去，不断消耗 CPU 的时钟周期，并使得 JVM 不能正常退出。

程序清单 7-2 一个仅运行一秒钟的素数生成器

```
List<BigInteger> aSecondOfPrimes() throws InterruptedException {
    PrimeGenerator generator = new PrimeGenerator();
    new Thread(generator).start();
    try {
        SECONDS.sleep(1);
    } finally {
        generator.cancel();
    }
    return generator.get();
}
```

一个可取消的任务必须拥有取消策略 (Cancellation Policy)，在这个策略中将详细地定义取消操作的“How”、“When”以及“What”，即其他代码如何（How）请求取消该任务，任务在何时（When）检查是否已经请求了取消，以及在响应取消请求时应该执行哪些（What）操作。

考虑现实世界中停止支付（Stop-Payment）支票的示例。银行通常都会规定如何提交一个停止支付的请求，在处理这些请求时需要做出哪些响应性保证，以及当支付中断后需要遵守哪些流程（例如通知该事务中涉及的其他银行，以及对付款人的账户进行费用评估）。这些流程和保证放在一起就构成了支票支付的取消策略。

PrimeGenerator 使用了一种简单的取消策略：客户代码通过调用 cancel 来请求取消，PrimeGenerator 在每次搜索素数前首先检查是否存在取消请求，如果存在则退出。

7.1.1 中断

PrimeGenerator 中的取消机制最终会使得搜索素数的任务退出，但在退出过程中需要花费一定的时间。然而，如果使用这种方法的任务调用了一个阻塞方法，例如 BlockingQueue.put，那么可能会产生一个更严重的问题——任务可能永远不会检查取消标志，因此永远不会结束。

在程序清单 7-3 中的 BrokenPrimeProducer 就说明了这个问题。生产者线程生成素数，并将它们放入一个阻塞队列。如果生产者的速度超过了消费者的处理速度，队列将被填满，put 方法也会阻塞。当生产者在 put 方法中阻塞时，如果消费者希望取消生产者任务，那么将发生什么情况？它可以调用 cancel 方法来设置 cancelled 标志，但此时生产者却永远不能检查这个标志，因为它无法从阻塞的 put 方法中恢复过来（因为消费者此时已经停止从队列中取出素数，所以 put 方法将一直保持阻塞状态）。

程序清单 7-3 不可靠的取消操作将把生产者置于阻塞的操作中（不要这么做）

```
class BrokenPrimeProducer extends Thread {
    private final BlockingQueue<BigInteger> queue;
    private volatile boolean cancelled = false;

    BrokenPrimeProducer(BlockingQueue<BigInteger> queue) {
        this.queue = queue;
    }

    public void run() {
        try {
```

```
            BigInteger p = BigInteger.ONE;
            while (!cancelled)
                queue.put(p = p.nextProbablePrime());
        } catch (InterruptedException consumed) { }
    }

    public void cancel() { cancelled = true;  }
}

void consumePrimes() throws InterruptedException {
    BlockingQueue<BigInteger> primes = ...;
    BrokenPrimeProducer producer = new BrokenPrimeProducer(primes);
    producer.start();
    try {
        while (needMorePrimes())
            consume(primes.take());
    } finally {
        producer.cancel();
    }
}
```

第 5 章曾提到，一些特殊的阻塞库的方法支持中断。线程中断是一种协作机制，线程可以通过这种机制来通知另一个线程，告诉它在合适的或者可能的情况下停止当前工作，并转而执行其他的工作。

> 在 Java 的 API 或语言规范中，并没有将中断与任何取消语义关联起来，但实际上，如果在取消之外的其他操作中使用中断，那么都是不合适的，并且很难支撑起更大的应用。

每个线程都有一个 boolean 类型的中断状态。当中断线程时，这个线程的中断状态将被设置为 true。在 Thread 中包含了中断线程以及查询线程中断状态的方法，如程序清单 7-4 所示。interrupt 方法能中断目标线程，而 isInterrupted 方法能返回目标线程的中断状态。静态的 interrupted 方法将清除当前线程的中断状态，并返回它之前的值，这也是清除中断状态的唯一方法。

程序清单 7-4　Thread 中的中断方法

```
public class Thread {
    public void interrupt() { ... }
    public boolean isInterrupted() { ... }
    public static boolean interrupted() { ... }
    ...
}
```

阻塞库方法，例如 Thread.sleep 和 Object.wait 等，都会检查线程何时中断，并且在发现中断时提前返回。它们在响应中断时执行的操作包括：清除中断状态，抛出 InterruptedException，表示阻塞操作由于中断而提前结束。JVM 并不能保证阻塞方法检测到中断的速度，但在实际情况中响应速度还是非常快的。

当线程在非阻塞状态下中断时，它的中断状态将被设置，然后根据将被取消的操作来检查中断状态以判断发生了中断。通过这样的方法，中断操作将变得“有黏性”——如果不触发 InterruptedException，那么中断状态将一直保持，直到明确地清除中断状态。

调用 interrupt 并不意味着立即停止目标线程正在进行的工作，而只是传递了请求中断的消息。

对中断操作的正确理解是：它并不会真正地中断一个正在运行的线程，而只是发出中断请求，然后由线程在下一个合适的时刻中断自己。（这些时刻也被称为取消点）。有些方法，例如 wait、sleep 和 join 等，将严格地处理这种请求，当它们收到中断请求或者在开始执行时发现某个已被设置好的中断状态时，将抛出一个异常。设计良好的方法可以完全忽略这种请求，只要它们能使调用代码对中断请求进行某种处理。设计糟糕的方法可能会屏蔽中断请求，从而导致调用栈中的其他代码无法对中断请求作出响应。

在使用静态的 interrupted 时应该小心，因为它会清除当前线程的中断状态。如果在调用 interrupted 时返回了 true，那么除非你想屏蔽这个中断，否则必须对它进行处理——可以抛出 InterruptedException，或者通过再次调用 interrupt 来恢复中断状态，如程序清单 5-10 所示。

BrokenPrimeProducer 说明了一些自定义的取消机制无法与可阻塞的库函数实现良好交互的原因。如果任务代码能够响应中断，那么可以使用中断作为取消机制，并且利用许多库类中提供的中断支持。

通常，中断是实现取消的最合理方式。

BrokenPrimeProducer 中的问题很容易解决（和简化）：使用中断而不是 boolean 标志来请求取消，如程序清单 7-5 所示。在每次迭代循环中，有两个位置可以检测出中断：在阻塞的 put 方法调用中，以及在循环开始处查询中断状态时。由于调用了阻塞的 put 方法，因此这里并不一定需要进行显式的检测，但执行检测却会使 PrimeProducer 对中断具有更高的响应性，因为它是在启动寻找素数任务之前检查中断的，而不是在任务完成之后。如果可中断的阻塞方法的调用频率并不高，不足以获得足够的响应性，那么显式地检测中断状态能起到一定的帮助作用。

程序清单 7-5 通过中断来取消

```
class PrimeProducer extends Thread {
    private final BlockingQueue<BigInteger> queue;

    PrimeProducer(BlockingQueue<BigInteger> queue) {
        this.queue = queue;
    }

    public void run() {
        try {
```

```
            BigInteger p = BigInteger.ONE;
            while (!Thread.currentThread().isInterrupted())
                queue.put(p = p.nextProbablePrime());
        } catch (InterruptedException consumed) {
            /*  允许线程退出  */
        }
    }
    public void cancel() { interrupt(); }
}
```

7.1.2 中断策略

正如任务中应该包含取消策略一样，线程同样应该包含中断策略。中断策略规定线程如何解释某个中断请求——当发现中断请求时，应该做哪些工作（如果需要的话），哪些工作单元对于中断来说是原子操作，以及以多快的速度来响应中断。

最合理的中断策略是某种形式的线程级 (Thread-Level) 取消操作或服务级 (Service-Level) 取消操作：尽快退出，在必要时进行清理，通知某个所有者该线程已经退出。此外还可以建立其他的中断策略，例如暂停服务或重新开始服务，但对于那些包含非标准中断策略的线程或线程池，只能用于能知道这些策略的任务中。

区分任务和线程对中断的反应是很重要的。一个中断请求可以有一个或多个接收者——中断线程池中的某个工作者线程，同时意味着“取消当前任务”和“关闭工作者线程”。

任务不会在其自己拥有的线程中执行，而是在某个服务（例如线程池）拥有的线程中执行。对于非线程所有者的代码来说（例如，对于线程池而言，任何在线程池实现以外的代码），应该小心地保存中断状态，这样拥有线程的代码才能对中断做出响应，即使“非所有者”代码也可以做出响应。（当你为一户人家打扫房屋时，即使主人不在，也不应该把在这段时间内收到的邮件扔掉，而应该把邮件收起来，等主人回来以后再交给他们处理，尽管你可以阅读他们的杂志。）

这就是为什么大多数可阻塞的库函数都只是抛出 InterruptedException 作为中断响应。它们永远不会在某个由自己拥有的线程中运行，因此它们为任务或库代码实现了最合理的取消策略：尽快退出执行流程，并把中断信息传递给调用者，从而使调用栈中的上层代码可以采取进一步的操作。

当检查到中断请求时，任务并不需要放弃所有的操作——它可以推迟处理中断请求，并直到某个更合适的时刻。因此需要记住中断请求，并在完成当前任务后抛出 InterruptedException 或者表示已收到中断请求。这项技术能够确保在更新过程中发生中断时，数据结构不会被破坏。

任务不应该对执行该任务的线程的中断策略做出任何假设，除非该任务被专门设计为在服务中运行，并且在这些服务中包含特定的中断策略。无论任务把中断视为取消，还是其他某个中断响应操作，都应该小心地保存执行线程的中断状态。如果除了将 InterruptedException 传递给调用者外还需要执行其他操作，那么应该在捕获 InterruptedException 之后恢复中断状态：

```
Thread.currentThread().interrupt();
```

正如任务代码不应该对其执行所在的线程的中断策略做出假设，执行取消操作的代码也不应该对线程的中断策略做出假设。线程应该只能由其所有者中断，所有者可以将线程的中断策略信息封装到某个合适的取消机制中，例如关闭 (shutdown) 方法。

由于每个线程拥有各自的中断策略，因此除非你知道中断对该线程的含义，否则就不应该中断这个线程。

批评者曾嘲笑 Java 的中断功能，因为它没有提供抢占式中断机制，而且还强迫开发人员必须处理 InterruptedException。然而，通过推迟中断请求的处理，开发人员能制定更灵活的中断策略，从而使应用程序在响应性和健壮性之间实现合理的平衡。

7.1.3 响应中断

在 5.4 节中，当调用可中断的阻塞函数时，例如 Thread.sleep 或 BlockingQueue.put 等，有两种实用策略可用于处理 InterruptedException：

- 传递异常（可能在执行某个特定于任务的清除操作之后），从而使你的方法也成为可中断的阻塞方法。
- 恢复中断状态，从而使调用栈中的上层代码能够对其进行处理。

传递 InterruptedException 与将 InterruptedException 添加到 throws 子句中一样容易，如程序清单 7-6 中的 getNextTask 所示。

程序清单 7-6 将 InterruptedException 传递给调用者

```
BlockingQueue<Task> queue;
...
public Task getNextTask() throws InterruptedException {
    return queue.take();
}
```

如果不想或无法传递 InterruptedException（或许通过 Runnable 来定义任务），那么需要寻找另一种方式来保存中断请求。一种标准的方法就是通过再次调用 interrupt 来恢复中断状态。你不能屏蔽 InterruptedException，例如在 catch 块中捕获到异常却不做任何处理，除非在你的代码中实现了线程的中断策略。虽然 PrimeProducer 屏蔽了中断，但这是因为它已经知道线程将要结束，因此在调用栈中已经没有上层代码需要知道中断信息。由于大多数代码并不知道它们将在哪个线程中运行，因此应该保存中断状态。

只有实现了线程中断策略的代码才可以屏蔽中断请求。在常规的任务和库代码中都不应该屏蔽中断请求。

对于一些不支持取消但仍可以调用可中断阻塞方法的操作，它们必须在循环中调用这些方

法，并在发现中断后重新尝试。在这种情况下，它们应该在本地保存中断状态，并在返回前恢复状态而不是在捕获 InterruptedException 时恢复状态，如程序清单 7-7 所示。如果过早地设置中断状态，就可能引起无限循环，因为大多数可中断的阻塞方法都会在入口处检查中断状态，并且当发现该状态已被设置时会立即抛出 InterruptedException。（通常，可中断的方法会在阻塞或进行重要的工作前首先检查中断，从而尽快地响应中断）。

程序清单 7-7　不可取消的任务在退出前恢复中断

```
public Task getNextTask(BlockingQueue<Taskgt; queue) {
    boolean interrupted = false;
    try {
        while (true) {
            try {
                return queue.take();
            } catch (InterruptedException e) {
                interrupted = true;
                // 重新尝试
            }
        }
    } finally {
        if (interrupted)
            Thread.currentThread().interrupt();
    }
}
```

如果代码不会调用可中断的阻塞方法，那么仍然可以通过在任务代码中轮询当前线程的中断状态来响应中断。要选择合适的轮询频率，就需要在效率和响应性之间进行权衡。如果响应性要求较高，那么不应该调用那些执行时间较长并且不响应中断的方法，从而对可调用的库代码进行一些限制。

在取消过程中可能涉及除了中断状态之外的其他状态。中断可以用来获得线程的注意，并且由中断线程保存的信息，可以为中断的线程提供进一步的指示。（当访问这些信息时，要确保使用同步。）

例如，当一个由 ThreadPoolExecutor 拥有的工作者线程检测到中断时，它会检查线程池是否正在关闭。如果是，它会在结束之前执行一些线程池清理工作，否则它可能创建一个新线程将线程池恢复到合理的规模。

7.1.4　示例：计时运行

许多问题永远也无法解决（例如，枚举所有的素数），而某些问题，能很快得到答案，也可能永远得不到答案。在这些情况下，如果能够指定“最多花 10 分钟搜索答案”或者“枚举出在 10 分钟内能找到的答案”，那么将是非常有用的。

程序清单 7-2 中的 aSecondOfPrimes 方法将启动一个 PrimeGenerator，并在 1 秒钟后中断。尽管 PrimeGenerator 可能需要超过 1 秒的时间才能停止，但它最终会发现中断，然后停止，并使线程结束。在执行任务时的另一个方面是，你希望知道在任务执行过程中是否会抛出异常。

如果 PrimeGenerator 在指定时限内抛出了一个未检查的异常，那么这个异常可能会被忽略，因为素数生成器在另一个独立的线程中运行，而这个线程并不会显式地处理异常。

在程序清单 7-8 中给出了在指定时间内运行一个任意的 Runnable 的示例。它在调用线程中运行任务，并安排了一个取消任务，在运行指定的时间间隔后中断它。这解决了从任务中抛出未检查异常的问题，因为该异常会被 timedRun 的调用者捕获。

程序清单 7-8 在外部线程中安排中断（不要这么做）

```
private static final ScheduledExecutorService cancelExec = ...;

public static void timedRun(Runnable r,
                            long timeout, TimeUnit unit) {
    final Thread taskThread = Thread.currentThread();
    cancelExec.schedule(new Runnable() {
        public void run() { taskThread.interrupt(); }
    }, timeout, unit);
    r.run();
}
```

这是一种非常简单的方法，但却破坏了以下规则：在中断线程之前，应该了解它的中断策略。由于 timedRun 可以从任意一个线程中调用，因此它无法知道这个调用线程的中断策略。如果任务在超时之前完成，那么中断 timedRun 所在线程的取消任务将在 timedRun 返回到调用者之后启动。我们不知道在这种情况下将运行什么代码，但结果一定是不好的。（可以使用 schedule 返回的 ScheduledFuture 来取消这个取消任务以避免这种风险，这种做法虽然可行，但却非常复杂。）

而且，如果任务不响应中断，那么 timedRun 会在任务结束时才返回，此时可能已经超过了指定的时限（或者还没有超过时限）。如果某个限时运行的服务没有在指定的时间内返回，那么将对调用者带来负面影响。

在程序清单 7-9 中解决了 aSecondOfPrimes 的异常处理问题以及之前解决方案中的问题。执行任务的线程拥有自己的执行策略，即使任务不响应中断，限时运行的方法仍能返回到它的调用者。在启动任务线程之后，timedRun 将执行一个限时的 join 方法。在 join 返回后，它将检查任务中是否有异常抛出，如果有的话，则会在调用 timedRun 的线程中再次抛出该异常。由于 Throwable 将在两个线程之间共享，因此该变量被声明为 volatile 类型，从而确保安全地将其从任务线程发布到 timedRun 线程。

程序清单 7-9 在专门的线程中中断任务

```
public static void timedRun(final Runnable r,
                            long timeout, TimeUnit unit)
                            throws InterruptedException {
    class RethrowableTask implements Runnable {
        private volatile Throwable t;
        public void run() {
            try { r.run(); }
            catch (Throwable t) { this.t = t; }
```

```
        }
        void rethrow() {
            if (t != null)
                throw launderThrowable(t);
        }
    }

    RethrowableTask task = new RethrowableTask();
    final Thread taskThread = new Thread(task);
    taskThread.start();
    cancelExec.schedule(new Runnable() {
        public void run() { taskThread.interrupt();  }
    }, timeout, unit);
    taskThread.join(unit.toMillis(timeout));
    task.rethrow();
}
```

在这个示例的代码中解决了前面示例中的问题，但由于它依赖于一个限时的 join，因此存在着 join 的不足：无法知道执行控制是因为线程正常退出而返回还是因为 join 超时而返回。[⊖]

7.1.5 通过 Future 来实现取消

我们已经使用了一种抽象机制来管理任务的生命周期，处理异常，以及实现取消，即 Future。通常，使用现有库中的类比自行编写更好，因此我们将继续使用 Future 和任务执行框架来构建 timedRun。

ExecutorService.submit 将返回一个 Future 来描述任务。Future 拥有一个 cancel 方法，该方法带有一个 boolean 类型的参数 mayInterruptIfRunning，表示取消操作是否成功。（这只是表示任务是否能够接收中断，而不是表示任务是否能检测并处理中断。）如果 mayInterruptIfRunning 为 true 并且任务当前正在某个线程中运行，那么这个线程能被中断。如果这个参数为 false，那么意味着“若任务还没有启动，就不要运行它”，这种方式应该用于那些不处理中断的任务中。

除非你清楚线程的中断策略，否则不要中断线程，那么在什么情况下调用 cancel 可以将参数指定为 true？执行任务的线程是由标准的 Executor 创建的，它实现了一种中断策略使得任务可以通过中断被取消，所以如果任务在标准 Executor 中运行，并通过它们的 Future 来取消任务，那么可以设置 mayInterruptIfRunning。当尝试取消某个任务时，不宜直接中断线程池，因为你并不知道当中断请求到达时正在运行什么任务——只能通过任务的 Future 来实现取消。这也是在编写任务时要将中断视为一个取消请求的另一个理由：可以通过任务的 Future 来取消它们。

程序清单 7-10 给出了另一个版本的 timedRun：将任务提交给一个 ExecutorService，并通过一个定时的 Future.get 来获得结果。如果 get 在返回时抛出了一个 TimeoutException，那么任务将通过它的 Future 来取消。（为了简化代码，这个版本的 timedRun 在 finally 块中将直接调用 Future.cancel，因为取消一个已完成的任务不会带来任何影响。）如果任务在被取消前就抛出一

⊖ 这是 Thread API 的一个缺陷，因为无论 join 是否成功地完成，在 Java 内存模型中都会有内存可见性结果，但 join 本身不会返回某个状态来表明它是否成功。

个异常，那么该异常将被重新抛出以便由调用者来处理异常。在程序清单 7-10 中还给出了另一种良好的编程习惯：取消那些不再需要结果的任务。（在程序清单 6-13 和程序清单 6-16 中使用了相同的技术。）

程序清单 7-10 通过 Future 来取消任务

```
public static void timedRun(Runnable r,
                            long timeout, TimeUnit unit)
                            throws InterruptedException {
    Future<?> task = taskExec.submit(r);
    try {
        task.get(timeout, unit);
    } catch (TimeoutException e) {
          // 接下来任务将被取消
    } catch (ExecutionException e) {
          //如果在任务中抛出了异常，那么重新抛出该异常
        throw launderThrowable(e.getCause());
    } finally {
          //如果任务已经结束，那么执行取消操作也不会带来任何影响
task.cancel(true);   // 如果任务正在运行，那么将被中断
    }
}
```

当 Future.get 抛出 InterruptedException 或 TimeoutException 时，如果你知道不再需要结果，那么就可以调用 Future.cancel 来取消任务。

7.1.6 处理不可中断的阻塞

在 Java 库中，许多可阻塞的方法都是通过提前返回或者抛出 InterruptedException 来响应中断请求的，从而使开发人员更容易构建出能响应取消请求的任务。然而，并非所有的可阻塞方法或者阻塞机制都能响应中断；如果一个线程由于执行同步的 Socket I/O 或者等待获得内置锁而阻塞，那么中断请求只能设置线程的中断状态，除此之外没有其他任何作用。对于那些由于执行不可中断操作而被阻塞的线程，可以使用类似于中断的手段来停止这些线程，但这要求我们必须知道线程阻塞的原因。

Java.io 包中的同步 Socket I/O。在服务器应用程序中，最常见的阻塞 I/O 形式就是对套接字进行读取和写入。虽然 InputStream 和 OutputStream 中的 read 和 write 等方法都不会响应中断，但通过关闭底层的套接字，可以使得由于执行 read 或 write 等方法而被阻塞的线程抛出一个 SocketException。

Java.io 包中的同步 I/O。当中断一个正在 InterruptibleChannel 上等待的线程时，将抛出 ClosedByInterruptException 并关闭链路（这还会使得其他在这条链路上阻塞的线程同样抛出 ClosedByInterruptException）。当关闭一个 InterruptibleChannel 时，将导致所有在链路操作上阻塞的线程都抛出 AsynchronousCloseException。大多数标准的 Channel 都实现了 InterruptibleChannel。

Selector 的异步 I/O。如果一个线程在调用 Selector.select 方法（在 java.nio.channels 中）时阻塞了，那么调用 close 或 wakeup 方法会使线程抛出 ClosedSelectorException 并提前返回。

获取某个锁。如果一个线程由于等待某个内置锁而阻塞，那么将无法响应中断，因为线程认为它肯定会获得锁，所以将不会理会中断请求。但是，在 Lock 类中提供了 lockInterruptibly 方法，该方法允许在等待一个锁的同时仍能响应中断，请参见第 13 章。

程序清单 7-11 的 ReaderThread 给出了如何封装非标准的取消操作。ReaderThread 管理了一个套接字连接，它采用同步方式从该套接字中读取数据，并将接收到的数据传递给 processBuffer。为了结束某个用户的连接或者关闭服务器，ReaderThread 改写了 interrupt 方法，使其既能处理标准的中断，也能关闭底层的套接字。因此，无论 ReaderThread 线程是在 read 方法中阻塞还是在某个可中断的阻塞方法中阻塞，都可以被中断并停止执行当前的工作。

程序清单 7-11　通过改写 interrupt 方法将非标准的取消操作封装在 Thread 中

```
public class ReaderThread extends Thread {
    private final Socket socket;
    private final InputStream in;

    public ReaderThread(Socket socket) throws IOException {
        this.socket = socket;
        this.in = socket.getInputStream();
    }

    public void  interrupt()  {
        try {
            socket.close();
        }
        catch (IOException ignored) { }
        finally {
            super.interrupt();
        }
    }

    public void run() {
        try {
            byte[] buf = new byte[BUFSZ];
            while (true) {
                int count = in.read(buf);
                if (count < 0)
                    break;
                else if (count > 0)
                    processBuffer(buf, count);
            }
        } catch (IOException e) { /* 允许线程退出 */  }
    }
}
```

7.1.7　采用 newTaskFor 来封装非标准的取消

我们可以通过 newTaskFor 方法来进一步优化 ReaderThread 中封装非标准取消的技术，这

是 Java 6 在 ThreadPoolExecutor 中的新增功能。当把一个 Callable 提交给 ExecutorService 时，submit 方法会返回一个 Future，我们可以通过这个 Future 来取消任务。newTaskFor 是一个工厂方法，它将创建 Future 来代表任务。newTaskFor 还能返回一个 RunnableFuture 接口，该接口扩展了 Future 和 Runnable（并由 FutureTask 实现）。

通过定制表示任务的 Future 可以改变 Future.cancel 的行为。例如，定制的取消代码可以实现日志记录或者收集取消操作的统计信息，以及取消一些不响应中断的操作。通过改写 interrupt 方法，ReaderThread 可以取消基于套接字的线程。同样，通过改写任务的 Future.cancel 方法也可以实现类似的功能。

在程序清单 7-12 的 CancellableTask 中定义了一个 CancellableTask 接口，该接口扩展了 Callable，并增加了一个 cancel 方法和一个 newTask 工厂方法来构造 RunnableFuture。CancellingExecutor 扩展了 ThreadPoolExecutor，并通过改写 newTaskFor 使得 CancellableTask 可以创建自己的 Future。

程序清单 7-12 通过 newTaskFor 将非标准的取消操作封装在一个任务中

```
public interface CancellableTask<T> extends Callable<T> {
    void cancel();
    RunnableFuture<T> newTask();
}

@ThreadSafe
public class CancellingExecutor extends ThreadPoolExecutor {
    ...
    protected<T> RunnableFuture<T> newTaskFor(Callable<T> callable) {
        if (callable instanceof CancellableTask)
            return ((CancellableTask<T>) callable).newTask();
        else
            return super.newTaskFor(callable);
    }
}

public abstract class SocketUsingTask<T>
        implements CancellableTask<T> {
    @GuardedBy("this") private Socket socket;

    protected synchronized void setSocket(Socket s) { socket = s; }

    public synchronized void cancel() {
        try {
            if (socket != null)
                socket.close();
        } catch (IOException ignored) { }
    }

    public RunnableFuture<T> newTask() {
        return new FutureTask<T>(this) {
            public boolean cancel(boolean mayInterruptIfRunning) {
                try {
                    SocketUsingTask.this.cancel();
                } finally {
```

```
                return super.cancel(mayInterruptIfRunning);
            }
        }
    };
    }
}
```

SocketUsingTask 实现了 CancellableTask，并定义了 Future.cancel 来关闭套接字和调用 super.cancel。如果 SocketUsingTask 通过其自己的 Future 来取消，那么底层的套接字将被关闭并且线程将被中断。因此它提高了任务对取消操作的响应性：不仅能够在调用可中断方法的同时确保响应取消操作，而且还能调用可阻调的套接字 I/O 方法。

7.2　停止基于线程的服务

应用程序通常会创建拥有多个线程的服务，例如线程池，并且这些服务的生命周期通常比创建它们的方法的生命周期更长。如果应用程序准备退出，那么这些服务所拥有的线程也需要结束。由于无法通过抢占式的方法来停止线程，因此它们需要自行结束。

正确的封装原则是：除非拥有某个线程，否则不能对该线程进行操控。例如，中断线程或者修改线程的优先级等。在线程 API 中，并没有对线程所有权给出正式的定义：线程由 Thread 对象表示，并且像其他对象一样可以被自由共享。然而，线程有一个相应的所有者，即创建该线程的类。因此线程池是其工作者线程的所有者，如果要中断这些线程，那么应该使用线程池。

与其他封装对象一样，线程的所有权是不可传递的：应用程序可以拥有服务，服务也可以拥有工作者线程，但应用程序并不能拥有工作者线程，因此应用程序不能直接停止工作者线程。相反，服务应该提供生命周期方法（Lifecycle Method）来关闭它自己以及它所拥有的线程。这样，当应用程序关闭该服务时，服务就可以关闭所有的线程了。在 ExecutorService 中提供了 shutdown 和 shutdownNow 等方法。同样，在其他拥有线程的服务中也应该提供类似的关闭机制。

对于持有线程的服务，只要服务的存在时间大于创建线程的方法的存在时间，那么就应该提供生命周期方法。

7.2.1　示例：日志服务

在大多数服务器应用程序中都会用到日志，例如，在代码中插入 println 语句就是一种简单的日志。像 PrintWriter 这样的字符流类是线程安全的，因此这种简单的方法不需要显式的同步[⊖]。然而，在 11.6 节中，我们将看到这种内联日志功能会给一些高容量的 (Highvolume) 应

⊖ 如果需要在单条日志消息中写入多行，那么要通过客户端加锁来避免多个线程不正确地交错输出。如果两个线程同时把多行栈追踪信息（Stack Trace）添加到同一个流中，并且每行信息对应一个 println 调用，那么这些信息在输出中将交错在一起，看上去就是一些虽然庞大但却毫无意义的栈追踪信息。

用程序带来一定的性能开销。另外一种替代方法是通过调用 log 方法将日志消息放入某个队列中，并由其他线程来处理。

在程序清单 7-13 的 LogWriter 中给出了一个简单的日志服务示例，其中日志操作在单独的日志线程中执行。产生日志消息的线程并不会将消息直接写入输出流，而是由 LogWriter 通过 BlockingQueue 将消息提交给日志线程，并由日志线程写入。这是一种多生产者单消费者（Multiple-Producer,Single-Consumer）的设计方式：每个调用 log 的操作都相当于一个生产者，而后台的日志线程则相当于消费者。如果消费者的处理速度低于生产者的生成速度，那么 BlockingQueue 将阻塞生产者，直到日志线程有能力处理新的日志消息。

程序清单 7-13 不支持关闭的生产者 – 消费者日志服务

```
public class LogWriter {
    private final BlockingQueue<String> queue;
    private final LoggerThread logger;

    public LogWriter(Writer writer) {
        this.queue = new LinkedBlockingQueue<String>(CAPACITY);
        this.logger = new LoggerThread(writer);
    }

    public void start() { logger.start(); }

    public void log(String msg) throws InterruptedException {
        queue.put(msg);
    }

    private class LoggerThread extends Thread {
        private final PrintWriter writer;
        ...
        public void run() {
            try {
                while (true)
                    writer.println(queue.take());
            } catch(InterruptedException ignored) {
            } finally {
                writer.close();
            }
        }
    }
}
```

为了使像 LogWriter 这样的服务在软件产品中能发挥实际的作用，还需要实现一种终止日志线程的方法，从而避免使 JVM 无法正常关闭。要停止日志线程是很容易的，因为它会反复调用 take，而 take 能响应中断。如果将日志线程修改为当捕获到 InterruptedException 时退出，那么只需中断日志线程就能停止服务。

然而，如果只是使日志线程退出，那么还不是一种完备的关闭机制。这种直接关闭的做法会丢失那些正在等待被写入到日志的信息，不仅如此，其他线程将在调用 log 时被阻塞，因为日志消息队列是满的，因此这些线程将无法解除阻塞状态。当取消一个生产者 - 消费者操作时，

需要同时取消生产者和消费者。在中断日志线程时会处理消费者，但在这个示例中，由于生产者并不是专门的线程，因此要取消它们将非常困难。

另一种关闭 LogWriter 的方法是：设置某个“已请求关闭”标志，以避免进一步提交日志消息，如程序清单 7-14 所示。在收到关闭请求后，消费者会把队列中的所有消息写入日志，并解除所有在调用 log 时阻塞的生产者。然而，在这个方法中存在着竞态条件问题，使得该方法并不可靠。log 的实现是一种“先判断再运行”的代码序列：生产者发现该服务还没有关闭，因此在关闭服务后仍然会将日志消息放入队列，这同样会使得生产者可能在调用 log 时阻塞并且无法解除阻塞状态。可以通过一些技巧来降低这种情况的发生概率（例如，在宣布队列被清空之前，让消费者等待数秒钟），但这些都没有解决问题的本质，即使很小的概率也可能导致程序发生故障。

程序清单 7-14　通过一种不可靠的方式为日志服务增加关闭支持

```
public void log(String msg) throws InterruptedException {
    if (!shutdownRequested)
        queue.put(msg);
    else
        throw new IllegalStateException("logger is shut down");
}
```

为 LogWriter 提供可靠关闭操作的方法是解决竞态条件问题，因而要使日志消息的提交操作成为原子操作。然而，我们不希望在消息加入队列时去持有一个锁，因为 put 方法本身就可以阻塞。我们采用的方法是：通过原子方式来检查关闭请求，并且有条件地递增一个计数器来“保持”提交消息的权利，如程序清单 7-15 中的 LogService 所示。

程序清单 7-15　向 LogWriter 添加可靠的取消操作

```
public class LogService {
    private final BlockingQueue<String> queue;
    private final LoggerThread loggerThread;
    private final PrintWriter writer;
    @GuardedBy("this") private boolean isShutdown;
    @GuardedBy("this") private int reservations;

    public void start() { loggerThread.start(); }

    public void stop() {
        synchronized (this) { isShutdown = true; }
        loggerThread.interrupt();
    }

    public void log(String msg) throws InterruptedException {
        synchronized (this) {
            if (isShutdown)
                throw new IllegalStateException(...);
            ++reservations;
        }
        queue.put(msg);
    }
```

```
    private class LoggerThread extends Thread {
        public void run() {
            try {
                while (true) {
                    try {
                        synchronized (LogService.this) {
                            if (isShutdown && reservations == 0)
                                break;
                        }
                        String msg = queue.take();
                        synchronized (LogService.this) { --reservations; }
                        writer.println(msg);
                    } catch (InterruptedException e) { /*  retry  */ }
                }
            } finally {
                writer.close();
            }
        }
    }
}
```

7.2.2 关闭 ExecutorService

在 6.2.4 节中，我们看到 ExecutorService 提供了两种关闭方法：使用 shutdown 正常关闭，以及使用 shutdownNow 强行关闭。在进行强行关闭时，shutdownNow 首先关闭当前正在执行的任务，然后返回所有尚未启动的任务清单。

这两种关闭方式的差别在于各自的安全性和响应性：强行关闭的速度更快，但风险也更大，因为任务很可能在执行到一半时被结束；而正常关闭虽然速度慢，但却更安全，因为 ExecutorService 会一直等到队列中的所有任务都执行完成后才关闭。在其他拥有线程的服务中也应该考虑提供类似的关闭方式以供选择。

简单的程序可以直接在 main 函数中启动和关闭全局的 ExecutorService。而在复杂程序中，通常会将 ExecutorService 封装在某个更高级别的服务中，并且该服务能提供其自己的生命周期方法，例如程序清单 7-16 中 LogService 的一种变化形式，它将管理线程的工作委托给一个 ExecutorService，而不是由其自行管理。通过封装 ExecutorService，可以将所有权链（Ownership Chain）从应用程序扩展到服务以及线程，所有权链上的各个成员都将管理它所拥有的服务或线程的生命周期。

程序清单 7-16 使用 ExecutorService 的日志服务

```
public class LogService {
    private final ExecutorService exec = newSingleThreadExecutor();
    ...
    public void start() { }

    public void stop() throws InterruptedException {
        try {
            exec.shutdown();
            exec.awaitTermination(TIMEOUT, UNIT);
        } finally {
```

```
            writer.close();
        }
    }
    public void log(String msg) {
        try {
            exec.execute(new WriteTask(msg));
        } catch (RejectedExecutionException ignored) { }
    }
}
```

7.2.3 “毒丸”对象

另一种关闭生产者－消费者服务的方式就是使用“毒丸 (Poison Pill)”对象：“毒丸”是指一个放在队列上的对象，其含义是：“当得到这个对象时，立即停止。”在 FIFO（先进先出）队列中，“毒丸”对象将确保消费者在关闭之前首先完成队列中的所有工作，在提交“毒丸”对象之前提交的所有工作都会被处理，而生产者在提交了“毒丸”对象后，将不会再提交任何工作。在程序清单 7-17、程序清单 7-18 和程序清单 7-19 中给出一个单生产者－单消费者的桌面搜索示例（来自程序清单 5-8），在这个示例中使用了“毒丸”对象来关闭服务。

程序清单 7-17　通过“毒丸”对象来关闭服务

```
public class IndexingService {
    private static final File POISON = new File("");
    private final IndexerThread consumer = new IndexerThread();
    private final CrawlerThread producer = new CrawlerThread();
    private final BlockingQueue<File> queue;
    private final FileFilter fileFilter;
    private final File root;

class CrawlerThread extends Thread { /* 程序清单 7-18*/ }
class IndexerThread extends Thread { /* 程序清单 7-19 */ }

    public void start() {
        producer.start();
        consumer.start();
    }

    public void stop() { producer.interrupt(); }

    public void awaitTermination() throws InterruptedException {
        consumer.join();
    }
}
```

程序清单 7-18　IndexingService 的生产者线程

```
public class CrawlerThread extends Thread {
    public void run() {
        try {
            crawl(root);
        } catch (InterruptedException e) { /*  发生异常  */  }
        finally {
```

```
            while (true) {
                try {
                    queue.put(POISON);
                    break;
                } catch (InterruptedException e1) { /* 重新尝试 */ }
            }
        }
    }

    private void crawl(File root) throws InterruptedException {
        ...
    }
}
```

程序清单 7-19 IndexingService 的消费者线程

```
public class IndexerThread extends Thread {
    public void run() {
        try {
            while (true) {
                File file = queue.take();
                if (file == POISON)
                   break;
                else
                   indexFile(file);
            }
        } catch (InterruptedException consumed) { }
    }
}
```

只有在生产者和消费者的数量都已知的情况下，才可以使用“毒丸”对象。在 IndexingService 中采用的解决方案可以扩展到多个生产者：只需每个生产者都向队列中放入一个“毒丸”对象，并且消费者仅当在接收到 $N_{producers}$ 个“毒丸”对象时才停止。这种方法也可以扩展到多个消费者的情况，只需生产者将 $N_{consumers}$ 个“毒丸”对象放入队列。然而，当生产者和消费者的数量较大时，这种方法将变得难以使用。只有在无界队列中，“毒丸”对象才能可靠地工作。

7.2.4 示例：只执行一次的服务

如果某个方法需要处理一批任务，并且当所有任务都处理完成后才返回，那么可以通过一个私有的 Executor 来简化服务的生命周期管理，其中该 Executor 的生命周期是由这个方法来控制的。（在这种情况下，invokeAll 和 invokeAny 等方法通常会起较大的作用。）

程序清单 7-20 中的 checkMail 方法能在多台主机上并行地检查新邮件。它创建一个私有的 Executor，并向每台主机提交一个任务。然后，当所有邮件检查任务都执行完成后，关闭 Executor 并等待结束。[⊖]

⊖ 之所以采用 AtomicBoolean 来代替 volatile 类型的 boolean，是因为能从内部的 Runnable 中访问 hasNewMail 标志，因此它必须是 final 类型以免被修改。

程序清单 7-20　使用私有的 Executor，并且该 Executor 的生命周期受限于方法调用

```
boolean checkMail(Set<String> hosts, long timeout, TimeUnit unit)
        throws InterruptedException {
    ExecutorService exec = Executors.newCachedThreadPool();
    final AtomicBoolean hasNewMail = new AtomicBoolean(false);
    try {
        for (final String host : hosts)
            exec.execute(new Runnable() {
                public void run() {
                    if (checkMail(host))
                        hasNewMail.set(true);
                }
            });
    } finally {
        exec.shutdown();
        exec.awaitTermination(timeout, unit);
    }
    return hasNewMail.get();
}
```

7.2.5　shutdownNow 的局限性

当通过 shutdownNow 来强行关闭 ExecutorService 时，它会尝试取消正在执行的任务，并返回所有已提交但尚未开始的任务，从而将这些任务写入日志或者保存起来以便之后进行处理。[⊖]

然而，我们无法通过常规方法来找出哪些任务已经开始但尚未结束。这意味着我们无法在关闭过程中知道正在执行的任务的状态，除非任务本身会执行某种检查。要知道哪些任务还没有完成，你不仅需要知道哪些任务还没有开始，而且还需要知道当 Executor 关闭时哪些任务正在执行。[⊜]

在程序清单 7-21 的 TrackingExecutor 中给出了如何在关闭过程中判断正在执行的任务。通过封装 ExecutorService 并使得 execute（类似地还有 submit，在这里没有给出）记录哪些任务是在关闭后取消的，TrackingExecutor 可以找出哪些任务已经开始但还没有正常完成。在 Executor 结束后，getCancelledTasks 返回被取消的任务清单。要使这项技术能发挥作用，任务在返回时必须维持线程的中断状态，在所有设计良好的任务中都会实现这个功能。

程序清单 7-21　在 ExecutorService 中跟踪在关闭之后被取消的任务

```
public class TrackingExecutor extends AbstractExecutorService {
    private final ExecutorService exec;
    private final Set<Runnable> tasksCancelledAtShutdown =
```

⊖ shutdownNow 返回的 Runnable 对象可能与提交给 ExecutorService 的 Runnable 对象并不相同：它们可能是被封装过的已提交任务。

⊜ 然而，在关闭过程中只会返回尚未开始的任务，而不会返回正在执行的任务。如果能返回所有这两种类型的任务，那么就不需要这种不确定的中间状态。

```
        Collections.synchronizedSet(new HashSet<Runnable>());
    ...
    public List<Runnable> getCancelledTasks() {
        if (!exec.isTerminated())
            throw new IllegalStateException(...);
        return new ArrayList<Runnable>(tasksCancelledAtShutdown);
    }

    public void execute(final Runnable runnable) {
        exec.execute(new Runnable() {
            public void run() {
                try {
                    runnable.run();
                } finally {
                    if (isShutdown()
                        && Thread.currentThread().isInterrupted())
                        tasksCancelledAtShutdown.add(runnable);
                }
            }
        });
    }

// 将 ExecutorService 的其他方法委托给 exec
}
```

在程序清单 7-22 的 WebCrawler 中给出了 TrackingExecutor 的用法。网页爬虫程序的工作通常是无穷尽的，因此当爬虫程序必须关闭时，我们通常希望保存它的状态，以便稍后重新启动。CrawlTask 提供了一个 getPage 方法，该方法能找出正在处理的页面。当爬虫程序关闭时，无论是还没有开始的任务，还是那些被取消的任务，都将记录它们的 URL，因此当爬虫程序重新启动时，就可以将这些 URL 的页面抓取任务加入到任务队列中。

程序清单 7-22 使用 TrackingExecutorService 来保存未完成的任务以备后续执行

```
public abstract class WebCrawler {
    private volatile TrackingExecutor exec;
    @GuardedBy("this")
    private final Set<URL> urlsToCrawl = new HashSet<URL>();
    ...
    public synchronized void start() {
        exec = new TrackingExecutor(
                Executors.newCachedThreadPool());
        for (URL url : urlsToCrawl) submitCrawlTask(url);
        urlsToCrawl.clear();
    }

    public synchronized void stop() throws InterruptedException {
        try {
            saveUncrawled(exec.shutdownNow());
            if (exec.awaitTermination(TIMEOUT, UNIT))
                saveUncrawled(exec.getCancelledTasks());
        } finally {
```

```
            exec = null;
        }
    }

    protected abstract List<URL> processPage(URL url);

    private void saveUncrawled(List<Runnable> uncrawled) {
        for (Runnable task : uncrawled)
            urlsToCrawl.add(((CrawlTask) task).getPage());
    }
    private void submitCrawlTask(URL u) {
        exec.execute(new CrawlTask(u));
    }
    private class CrawlTask implements Runnable {
        private final URL url;
        ...
        public void run() {
            for (URL link : processPage(url)) {
                if (Thread.currentThread().isInterrupted())
                    return;
                submitCrawlTask(link);
            }
        }
        public URL getPage() { return url; }
    }
}
```

在 TrackingExecutor 中存在一个不可避免的竞态条件，从而产生“误报”问题：一些被认为已取消的任务实际上已经执行完成。这个问题的原因在于，在任务执行最后一条指令以及线程池将任务记录为“结束”的两个时刻之间，线程池可能被关闭。如果任务是幂等的（Idempotent，即将任务执行两次与执行一次会得到相同的结果），那么这不会存在问题，在网页爬虫程序中就是这种情况。否则，在应用程序中必须考虑这种风险，并对“误报”问题做好准备。

7.3 处理非正常的线程终止

当单线程的控制台程序由于发生了一个未捕获的异常而终止时，程序将停止运行，并产生与程序正常输出非常不同的栈追踪信息，这种情况是很容易理解的。然而，如果并发程序中的某个线程发生故障，那么通常并不会如此明显。在控制台中可能会输出栈追踪信息，但没有人会观察控制台。此外，当线程发生故障时，应用程序可能看起来仍然在工作，所以这个失败很可能会被忽略。幸运的是，我们有可以监测并防止在程序中“遗漏”线程的方法。

导致线程提前死亡的最主要原因就是 RuntimeException。由于这些异常表示出现了某种编程错误或者其他不可修复的错误，因此它们通常不会被捕获。它们不会在调用栈中逐层传递，而是默认地在控制台中输出栈追踪信息，并终止线程。

线程非正常退出的后果可能是良性的，也可能是恶性的，这要取决于线程在应用程序中的作用。虽然在线程池中丢失一个线程可能会对性能带来一定影响，但如果程序能在包含 50 个

线程的线程池上运行良好，那么在包含 49 个线程的线程池上通常也能运行良好。然而，如果在 GUI 程序中丢失了事件分派线程，那么造成的影响将非常显著——应用程序将停止处理事件并且 GUI 会因此失去响应。在第 6 章的 OutOfTime 中给出了由于遗漏线程而造成的严重后果：Timer 表示的服务将永远无法使用。

任何代码都可能抛出一个 RuntimeException。每当调用另一个方法时，都要对它的行为保持怀疑，不要盲目地认为它一定会正常返回，或者一定会抛出在方法原型中声明的某个已检查异常。对调用的代码越不熟悉，就越应该对其代码行为保持怀疑。

在任务处理线程（例如线程池中的工作者线程或者 Swing 的事件派发线程等）的生命周期中，将通过某种抽象机制（例如 Runnable）来调用许多未知的代码，我们应该对在这些线程中执行的代码能否表现出正确的行为保持怀疑。像 Swing 事件线程这样的服务可能只是因为某个编写不当的事件处理器抛出 NullPointerException 而失败，这种情况是非常糟糕的。因此，这些线程应该在 try-catch 代码块中调用这些任务，这样就能捕获那些未检查的异常了，或者也可以使用 try-finally 代码块来确保框架能够知道线程非正常退出的情况，并做出正确的响应。在这种情况下，你或许会考虑捕获 RuntimeException，即当通过 Runnable 这样的抽象机制来调用未知的和不可信的代码时。[⊖]

在程序清单 7-23 中给出了如何在线程池内部构建一个工作者线程。如果任务抛出了一个未检查异常，那么它将使线程终结，但会首先通知框架该线程已经终结。然后，框架可能会用新的线程来代替这个工作线程，也可能不会，因为线程池正在关闭，或者当前已有足够多的线程能满足需要。ThreadPoolExecutor 和 Swing 都通过这项技术来确保行为糟糕的任务不会影响到后续任务的执行。当编写一个向线程池提交任务的工作者线程类时，或者调用不可信的外部代码时（例如动态加载的插件），使用这些方法中的某一种可以避免某个编写得糟糕的任务或插件不会影响调用它的整个线程。

程序清单 7-23 典型的线程池工作者线程结构

```
public void run() {
    Throwable thrown = null;
    try {
        while (!isInterrupted())
            runTask(getTaskFromWorkQueue());
    } catch (Throwable e) {
        thrown = e;
    } finally {
        threadExited(this, thrown);
    }
}
```

⊖ 这项技术的安全性存在着一些争议。当线程抛出一个未检查异常时，整个应用程序都可能受到影响。但其替代方法——关闭整个应用程序，通常是更不切实际的。

未捕获异常的处理

上节介绍了一种主动方法来解决未检查异常。在 Thread API 中同样提供了 UncaughtExceptionHandler，它能检测出某个线程由于未捕获的异常而终结的情况。这两种方法是互补的，通过将二者结合在一起，就能有效地防止线程泄漏问题。

当一个线程由于未捕获异常而退出时，JVM 会把这个事件报告给应用程序提供的 UncaughtExceptionHandler 异常处理器（见程序清单 7-24）。如果没有提供任何异常处理器，那么默认的行为是将栈追踪信息输出到 System.err。[⊖]

程序清单 7-24　UncaughtExceptionHandler 接口

```
public interface UncaughtExceptionHandler {
    void uncaughtException(Thread t, Throwable e);
}
```

异常处理器如何处理未捕获异常，取决于对服务质量的需求。最常见的响应方式是将一个错误信息以及相应的栈追踪信息写入应用程序日志中，如程序清单 7-25 所示。异常处理器还可以采取更直接的响应，例如尝试重新启动线程，关闭应用程序，或者执行其他修复或诊断等操作。

程序清单 7-25　将异常写入日志的 UncaughtExceptionHandler

```
public class UEHLogger implements Thread.UncaughtExceptionHandler {
    public void uncaughtException(Thread t, Throwable e) {
        Logger logger = Logger.getAnonymousLogger();
        logger.log(Level.SEVERE,
              "Thread terminated with exception: " + t.getName(),
              e);
    }
}
```

在运行时间较长的应用程序中，通常会为所有线程的未捕获异常指定同一个异常处理器，并且该处理器至少会将异常信息记录到日志中。

要为线程池中的所有线程设置一个 UncaughtExceptionHandler，需要为 ThreadPoolExecutor 的构造函数提供一个 ThreadFactory。（与所有的线程操控一样，只有线程的所有者

⊖ 在 Java 5.0 之前，控制 UncaughtExceptionHandler 的唯一方法就是对 ThreadGroup 进行子类化。在 Java 5.0 及之后的版本中，可以通过 Thread.setUncaughtExceptionHandler 为每个线程设置一个 UncaughtExceptionHandler，还可以使用 setDefaultUncaughtExceptionHandler 来设置默认的 UncaughtExceptionHandler。然而，在这些异常处理器中，只有其中一个将被调用——JVM 首先搜索每个线程的异常处理器，然后再搜索一个 ThreadGroup 的异常处理器。ThreadGroup 中的默认异常处理器实现将异常处理工作逐层委托给它的上层 ThreadGroup，直至其中某个 ThreadGroup 的异常处理器能够处理该未捕获异常，否则将一直传递到顶层的 ThreadGroup。顶层 ThreadGroup 的异常处理器委托给默认的系统处理器（如果存在，在默认情况下为空），否则将把栈追踪信息输出到控制台。

能够改变线程的 UncaughtExceptionHandler。）标准线程池允许当发生未捕获异常时结束线程，但由于使用了一个 try-finally 代码块来接收通知，因此当线程结束时，将有新的线程来代替它。如果没有提供捕获异常处理器或者其他的故障通知机制，那么任务会悄悄失败，从而导致极大的混乱。如果你希望在任务由于发生异常而失败时获得通知，并且执行一些特定于任务的恢复操作，那么可以将任务封装在能捕获异常的 Runnable 或 Callable 中，或者改写 ThreadPoolExecutor 的 afterExecute 方法。

令人困惑的是，只有通过 execute 提交的任务，才能将它抛出的异常交给未捕获异常处理器，而通过 submit 提交的任务，无论是抛出的未检查异常还是已检查异常，都将被认为是任务返回状态的一部分。如果一个由 submit 提交的任务由于抛出了异常而结束，那么这个异常将被 Future.get 封装在 ExecutionException 中重新抛出。

7.4 JVM 关闭

JVM 既可以正常关闭，也可以强行关闭。正常关闭的触发方式有多种，包括：当最后一个“正常（非守护）”线程结束时，或者当调用了 System.exit 时，或者通过其他特定于平台的方法关闭时（例如发送了 SIGINT 信号或键入 Ctrl-C）。虽然可以通过这些标准方法来正常关闭 JVM，但也可以通过调用 Runtime. halt 或者在操作系统中“杀死”JVM 进程（例如发送 SIGKILL）来强行关闭 JVM。

7.4.1 关闭钩子

在正常关闭中，JVM 首先调用所有已注册的关闭钩子（Shutdown Hook）。关闭钩子是指通过 Runtime.addShutdownHook 注册的但尚未开始的线程。JVM 并不能保证关闭钩子的调用顺序。在关闭应用程序线程时，如果有（守护或非守护）线程仍然在运行，那么这些线程接下来将与关闭进程并发执行。当所有的关闭钩子都执行结束时，如果 runFinalizersOnExit 为 true，那么 JVM 将运行终结器，然后再停止。JVM 并不会停止或中断任何在关闭时仍然运行的应用程序线程。当 JVM 最终结束时，这些线程将被强行结束。如果关闭钩子或终结器没有执行完成，那么正常关闭进程“挂起”并且 JVM 必须被强行关闭。当被强行关闭时，只是关闭 JVM，而不会运行关闭钩子。

关闭钩子应该是线程安全的：它们在访问共享数据时必须使用同步机制，并且小心地避免发生死锁，这与其他并发代码的要求相同。而且，关闭钩子不应该对应用程序的状态（例如，其他服务是否已经关闭，或者所有的正常线程是否已经执行完成）或者 JVM 的关闭原因做出任何假设，因此在编写关闭钩子的代码时必须考虑周全。最后，关闭钩子必须尽快退出，因为它们会延迟 JVM 的结束时间，而用户可能希望 JVM 能尽快终止。

关闭钩子可以用于实现服务或应用程序的清理工作，例如删除临时文件，或者清除无法由操作系统自动清除的资源。在程序清单 7-26 中给出了如何使程序清单 7-16 中的 LogService 在其 start 方法中注册一个关闭钩子，从而确保在退出时关闭日志文件。

由于关闭钩子将并发执行，因此在关闭日志文件时可能导致其他需要日志服务的关闭钩子

产生问题。为了避免这种情况，关闭钩子不应该依赖那些可能被应用程序或其他关闭钩子关闭的服务。实现这种功能的一种方式是对所有服务使用同一个关闭钩子（而不是每个服务使用一个不同的关闭钩子），并且在该关闭钩子中执行一系列的关闭操作。这确保了关闭操作在单个线程中串行执行，从而避免了在关闭操作之间出现竞态条件或死锁等问题。无论是否使用关闭钩子，都可以使用这项技术，通过将各个关闭操作串行执行而不是并行执行，可以消除许多潜在的故障。当应用程序需要维护多个服务之间的显式依赖信息时，这项技术可以确保关闭操作按照正确的顺序执行。

程序清单 7-26　通过注册一个关闭钩子来停止日志服务

```
public void start() {
    Runtime.getRuntime().addShutdownHook(new Thread() {
        public void run() {
            try { LogService.this.stop(); }
            catch (InterruptedException ignored) {}
        }
    });
}
```

7.4.2　守护线程

有时候，你希望创建一个线程来执行一些辅助工作，但又不希望这个线程阻碍 JVM 的关闭。在这种情况下就需要使用守护线程 (Daemon Thread)。

线程可分为两种：普通线程和守护线程。在 JVM 启动时创建的所有线程中，除了主线程以外，其他的线程都是守护线程（例如垃圾回收器以及其他执行辅助工作的线程）。当创建一个新线程时，新线程将继承创建它的线程的守护状态，因此在默认情况下，主线程创建的所有线程都是普通线程。

普通线程与守护线程之间的差异仅在于当线程退出时发生的操作。当一个线程退出时，JVM 会检查其他正在运行的线程，如果这些线程都是守护线程，那么 JVM 会正常退出操作。当 JVM 停止时，所有仍然存在的守护线程都将被抛弃——既不会执行 finally 代码块，也不会执行回卷栈，而 JVM 只是直接退出。

我们应尽可能少地使用守护线程——很少有操作能够在不进行清理的情况下被安全地抛弃。特别是，如果在守护线程中执行可能包含 I/O 操作的任务，那么将是一种危险的行为。守护线程最好用于执行“内部”任务，例如周期性地从内存的缓存中移除逾期的数据。

此外，守护线程通常不能用来替代应用程序管理程序中各个服务的生命周期。

7.4.3　终结器

当不再需要内存资源时，可以通过垃圾回收器来回收它们，但对于其他一些资源，例如文件句柄或套接字句柄，当不再需要它们时，必须显式地交还给操作系统。为了实现这个功能，

垃圾回收器对那些定义了 finalize 方法的对象会进行特殊处理：在回收器释放它们后，调用它们的 finalize 方法，从而保证一些持久化的资源被释放。

由于终结器可以在某个由 JVM 管理的线程中运行，因此终结器访问的任何状态都可能被多个线程访问，这样就必须对其访问操作进行同步。终结器并不能保证它们将在何时运行甚至是否会运行，并且复杂的终结器通常还会在对象上产生巨大的性能开销。要编写正确的终结器是非常困难的[⊖]。在大多数情况下，通过使用 finally 代码块和显式的 close 方法，能够比使用终结器更好地管理资源。唯一的例外情况在于：当需要管理对象，并且该对象持有的资源是通过本地方法获得的。基于这些原因以及其他一些原因，我们要尽量避免编写或使用包含终结器的类（除非是平台库中的类）[EJ Item 6]。

避免使用终结器。

小结

在任务、线程、服务以及应用程序等模块中的生命周期结束问题，可能会增加它们在设计和实现时的复杂性。Java 并没有提供某种抢占式的机制来取消操作或者终结线程。相反，它提供了一种协作式的中断机制来实现取消操作，但这要依赖于如何构建取消操作的协议，以及能否始终遵循这些协议。通过使用 FutureTask 和 Executor 框架，可以帮助我们构建可取消的任务和服务。

⊖ 请参阅（Boehm，2005）并了解在编写终结器时存在的各种困难。